# Lecture 1 - How Professional MCU Drivers Are Designed

Bare-Metal STM32 Driver Development Course

Target: STM32L452RE (Cortex-M4) — NUCLEO-L452RE

---

# Lecture 1 — How Professional MCU Drivers Are Designed

**Course:** Bare-Metal STM32 Driver Development **Target MCU:** STM32L452RE — ARM Cortex-M4 **Board:** NUCLEO-L452RE **Prerequisite Knowledge:** C programming, basic GPIO concepts **Reference:** STM32L4 Series Reference Manual (RM0394) — used for context only, never copied verbatim

---

## 1. Learning Objectives

By the end of this lecture, you will be able to:

| # | Objective | Why It Matters | Where Used in Industry |
|---|---|---|---|
| 1 | Define what a "device driver" actually is in embedded systems | Removes the vague, hand-wavy idea of "driver = code that talks to hardware" | Every embedded product — medical, automotive, industrial |
| 2 | Explain the full embedded software stack from silicon to application | Lets you reason about *where* a bug or design decision belongs | Used when architecting any firmware project |
| 3 | Understand why raw register access must be hidden behind APIs | Prevents fragile, unmaintainable "spaghetti register code" | STM32Cube, Zephyr, Linux, AUTOSAR |
| 4 | Understand CMSIS and why ARM created it | CMSIS is the *foundation* every STM32 driver you write will sit on | All ARM Cortex-M vendors (ST, NXP, TI, Nordic, etc.) |
| 5 | Compare driver architectures across STM32Cube, Zephyr, Linux, and automotive stacks | Gives you the vocabulary and mental models used by senior firmware engineers | Job interviews, code reviews, architecture design docs |
| 6 | Learn the core design principles: abstraction, modularity, encapsulation, loose coupling, high cohesion | These are *not* academic buzzwords — they are the actual reasons professional drivers look the way they do | Every well-engineered SDK you will ever read |
| 7 | Learn the driver lifecycle (create → init → configure → operate → interrupt → deinit → power management → recovery) | This lifecycle is the skeleton every driver you write in this course will follow | I2C, SPI, UART, ADC, Timer drivers — all of them |
| 8 | Design (not yet fully implement) a tiny LED driver using professional architecture | Bridges theory into a concrete, small, understandable example before we tackle I2C | Foundational pattern for every peripheral driver |

**Why this lecture exists before we touch I2C:** If we jump straight into I2C registers, you will learn *I2C*, but you will not learn how to design *drivers*. Every peripheral driver you build for the rest of this course (I2C, SPI, UART, Timers, ADC, DMA) will reuse the exact architectural skeleton introduced here. This lecture is the skeleton. Later lectures put muscle on the bones.

---

## 2. Prerequisites

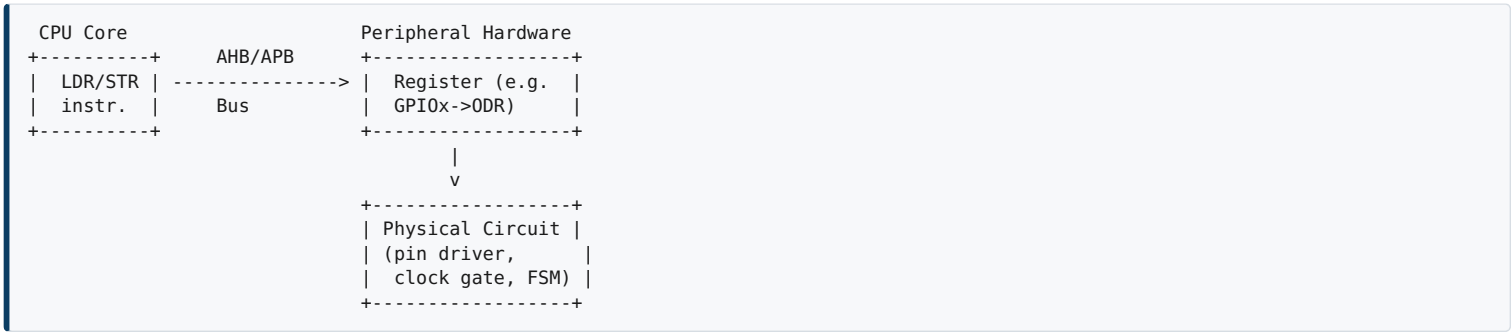
Before continuing, you should be comfortable with:

- **C language:** structs, pointers, function pointers, `typedef`, `enum`, bitwise operators (`|`, `&`, `~`, `<<`, `>>`), the `volatile` keyword, header/source file separation.
- **Basic GPIO knowledge:** you should know that setting a bit in a register can turn an LED on, and that this happens because the register is mapped to a real physical circuit inside the MCU (a flip-flop driving a pin).

- **Basic number systems:** hexadecimal and binary representation of register values.
- **Basic build process awareness:** you don't need deep linker knowledge yet — Part 2 will re-teach what you need.

### Quick Revision: What is a Register, Really?

A register is **not** a variable. A variable lives in RAM and only affects software state. A register lives at a fixed physical address inside the MCU's peripheral hardware block, and writing to it **physically changes the state of a hardware circuit** — a clock gate opens, a pin's output driver switches on, a state machine inside the I2C peripheral moves to a new state.




This distinction — **register access is a physical action, not just a software one** — is the single most important mental model for this entire course. Every driver we write exists to manage this physical action safely and predictably.

---

## 3. Part 1 — Introduction to Device Drivers

### 3.1 What Is a Device Driver?

A **device driver** is a piece of software whose sole responsibility is to control a specific piece of hardware **on behalf of** higher-level software, by translating simple, meaningful function calls into the exact sequence of register reads/writes that the hardware requires.

Formally, a driver has three defining properties:

1. **It owns the hardware.** No other software module is allowed to touch that peripheral's registers directly.
2. **It exposes a stable API.** The rest of the system calls functions like `I2C_MasterTransmit()`, not `I2C1->CR2 = ...`.
3. **It hides hardware-specific detail.** The caller does not need to know which bits mean what, or which silicon errata require a workaround.

> ***Common Misconception:*** *A driver is not "the code that uses the registers." A print statement that writes directly to `GPIOA->ODR` from inside `main()` is **not** a driver — it is unmanaged register access with no ownership, no API, and no abstraction. A driver requires architecture, not just register writes.*

### 3.2 Why Device Drivers Exist

Without drivers, every application programmer would need to:

- Memorize exact register addresses and bit positions for every peripheral, on every chip revision.
- Re-discover chip errata (silicon bugs) independently.
- Re-implement identical logic (e.g., "wait for a flag, then write data") in every project.
- Risk breaking hardware state because two unrelated pieces of code both write to the same register without knowing about each other.

Drivers exist to solve exactly these problems, by concentrating **hardware knowledge** into one well-tested module, and exposing **behavior**, not **registers**.

### 3.3 The Embedded Software Stack

```
+-------------------------------------------------------------+
|                     APPLICATION LAYER                       |
|   (business logic: "read temperature every 1 second")       |
+-------------------------------------------------------------+
|                     MIDDLEWARE LAYER                        |
|   (protocol stacks, filesystem, USB stack, RTOS services)   |
+-------------------------------------------------------------+
|                   PERIPHERAL DRIVER LAYER                   |
|   (I2C_Driver, SPI_Driver, UART_Driver, GPIO_Driver ...)    |
+-------------------------------------------------------------+
|                        CMSIS LAYER                          |
|   (register structs, core intrinsics, startup, interrupts)  |
+-------------------------------------------------------------+
|                      STARTUP CODE                           |
|   (stack pointer init, .data/.bss init, vector table setup) |
+-------------------------------------------------------------+
|                      LINKER SCRIPT                          |
|   (defines memory regions: FLASH, SRAM, sections)           |
+-------------------------------------------------------------+
|                   ARM CORTEX-M4 PROCESSOR                   |
|   (fetch-decode-execute, NVIC, SysTick, bus matrix)         |
+-------------------------------------------------------------+
|                         HARDWARE                            |
|   (I2C peripheral silicon, GPIO pads, clock trees, flash)   |
+-------------------------------------------------------------+
```

Data and control flow **downward** when the application requests an action, and interrupts flow **upward** when hardware needs attention. We dedicate all of Part 2 to explaining every layer in this diagram.

## 3.4 Hardware vs Software

| Aspect | Hardware | Software |
|---|---|---|
| What it is | Physical silicon: transistors, registers, clock trees, pads | Instructions executed by the CPU core |
| How it's changed | Fixed at manufacture (fab process) | Can be recompiled and reflashed |
| How firmware affects it | By writing to memory-mapped registers | By writing to variables in RAM |
| Failure mode | Physical damage, electrical noise, silicon errata | Logic bugs, race conditions, memory corruption |
| Who "owns" it in a well-designed system | The driver module | N/A — software has no ownership, only the driver mediates |

## 3.5 Firmware vs Driver vs Application

These three words are used loosely in casual conversation, but they mean specific things architecturally:

- **Firmware** is the *entire* compiled binary that runs on the MCU — it is the umbrella term including startup code, drivers, middleware, and application.
- **Driver** is *one module inside* the firmware, responsible for exactly one peripheral (or one class of peripheral).
- **Application** is the top-level logic that decides *what* should happen ("turn the pump on when temperature exceeds 80°C"), and calls drivers to make it happen — it does not know or care *how* the pump is switched on at the register level.

```
FIRMWARE (the whole .bin/.elf file)
├── Startup Code
├── CMSIS
├── Drivers/          <-- one driver per peripheral
│    ├── gpio_driver.c
│    ├── i2c_driver.c
│    └── uart_driver.c
├── Middleware/        <-- optional, e.g., FatFS, LWIP
└── Application/
     └── main.c        <-- calls driver APIs only
```

## 3.6 Why Applications Should Never Access Registers Directly

Imagine two engineers on the same team. Engineer A writes application code that directly sets `I2C1->CR1 |= I2C_CR1_PE;` inside a sensor-reading task. Engineer B, months later, writes a second task that also directly manipulates `I2C1->CR1` to reconfigure the same peripheral for a different sensor. Neither engineer knows about the other's code.

**Consequences of direct register access from applications:**

1. **Loss of ownership** — two pieces of code can conflict over hardware state without any compiler warning.
2. **Loss of portability** — if the project moves from STM32L452 to STM32F407, every `main.c` call site referencing raw registers must be found and rewritten.
3. **Loss of testability** — you cannot unit-test application logic on a PC if it directly touches memory-mapped hardware addresses that don't exist on a PC.

4. **Loss of maintainability** — a bug fix (e.g., a required delay after enabling a peripheral) must now be duplicated in every call site instead of fixed once inside the driver.
5. **Silicon errata become everyone's problem** — if bit 9 of a status register has a hardware quirk (very common in real MCUs), a driver author can work around it in *one place*. Direct-access code means every application developer must independently know and defend against that quirk.

This is why professional firmware places a **hard architectural boundary** between application code and hardware registers, and that boundary *is* the driver.

### 3.7 Driver Responsibilities

A professionally designed driver is responsible for:

| Responsibility | Explanation |
|---|---|
| **Initialization** | Bringing the peripheral from reset state to a known, configured, ready state |
| **Configuration** | Applying user-selected parameters (e.g., I2C speed, GPIO mode) validly |
| **Data transfer** | Performing the actual read/write operation the peripheral exists for |
| **Status/error reporting** | Translating raw status/error bits into a clean, documented return code |
| **Interrupt handling** | Servicing hardware events promptly and safely, and notifying upper layers |
| **State tracking** | Knowing whether the peripheral is idle, busy, or in an error state, and refusing invalid operations |
| **Resource protection** | Preventing concurrent, conflicting access to the same peripheral instance |
| **De-initialization / power-down** | Cleanly disabling the peripheral and its clock when no longer needed |

### 3.8 The Four Pillars: Abstraction, Portability, Maintainability, Scalability

These four words appear in almost every professional embedded job description, and this course treats them as *engineering requirements*, not marketing language.

- **Abstraction** — hiding *how* something is done behind a description of *what* is done. `I2C_MasterTransmit(handle, addr, data, len)` is abstraction: the caller does not see start-condition generation, ACK polling, or clock stretching.
- **Portability** — the ability to reuse the same driver (or the same driver *interface*) across different MCUs, or even different vendors, by isolating hardware-specific code behind a stable API.
- **Maintainability** — the ability for a different engineer (or you, six months later) to fix a bug or add a feature without having to re-understand the entire codebase.
- **Scalability** — the ability for the codebase to grow (more peripherals, more instances, more features) without an explosion in complexity or duplicated code.

> ***Engineering Insight:*** *These four properties are not independent — they reinforce each other. Abstraction enables portability. Portability forces you into modularity, which improves maintainability. Maintainability is what makes scalability safe rather than reckless.*

---

## 4. Part 2 — Complete Embedded Software Architecture

We now explain each layer of the stack shown in Section 3.3, from the top down, and then trace how a single function call travels all the way down to hardware and back.

### 4.1 Application Layer

The application layer contains **business logic** — the reason the product exists. Example: "every 5 seconds, read the ambient temperature sensor over I2C, and if it exceeds a threshold, trigger a fan relay." The application layer:

- Calls driver APIs ( `I2C_MasterReceive(...)` , `GPIO_WritePin(...)` ).
- Contains no register addresses, no bit masks, no peripheral-specific constants.
- Should be almost entirely portable to a *different MCU* if the driver API is preserved.

## 4.2 Middleware Layer

Middleware sits between application and drivers, and provides **reusable services** that are not peripheral-specific by themselves, but are built on top of one or more drivers. Examples: a filesystem (built on top of an SPI/SDIO driver), a USB stack, an RTOS's task scheduler, a Modbus protocol stack (built on top of a UART driver). Not every project needs middleware — a simple bare-metal blink project has none — but as complexity grows, middleware prevents application code from re-implementing protocol logic.

## 4.3 Peripheral Driver Layer

This is the layer this course focuses on building. Each driver:

- Manages exactly one peripheral **type** (I2C, SPI, UART, GPIO, Timer…).
- Supports multiple **instances** of that peripheral (STM32L452 has I2C1, I2C2, I2C3) through a handle-based design (explained in Part 5).
- Sits directly on top of CMSIS register definitions.

## 4.4 CMSIS Layer

CMSIS is explained in full depth in Part 3. For now: CMSIS is the layer that gives your driver code a **named, structured, standardized way** to refer to registers, instead of raw hex addresses.

## 4.5 Startup Code

Startup code is the very first code that executes when the MCU exits reset, **before `main()` runs**. Its responsibilities:

1. Set the initial stack pointer (read from the vector table).
2. Copy initialized global/static variables (`.data` section) from Flash into RAM.
3. Zero-fill uninitialized global/static variables (`.bss` section).
4. Optionally call `SystemInit()` (a CMSIS-defined function that configures clocks very early).
5. Call `main()`.

```
RESET
  |
  v
Read initial SP from vector table[0]
  |
  v
Read initial PC (Reset_Handler address) from vector table[1]
  |
  v
Jump to Reset_Handler
  |
  ├── Copy .data from FLASH -> SRAM
  ├── Zero-fill .bss in SRAM
  ├── Call SystemInit()
  └── Call main()
```

## 4.6 Linker Script

The linker script does not execute — it is a **build-time description of memory**. It tells the linker:

- Where FLASH begins and how large it is (e.g., `0x08000000`, 512 KB on STM32L452RE).
- Where SRAM begins and how large it is (e.g., `0x20000000`, 160 KB on STM32L452RE).
- Where each section (`.text`, `.data`, `.bss`, `.rodata`) should be placed.
- Where the stack and heap live.

```
Memory Map (STM32L452RE — conceptual, not exhaustive)

0x0800_0000 +----------------------+
            |   FLASH (512 KB)     |   .text, .rodata, .data (init values)
0x0807_FFFF +----------------------+
            |                      |
0x2000_0000 +----------------------+
            |   SRAM1 (~ 160 KB)   |   .data (runtime), .bss, heap, stack
0x2002_7FFF +----------------------+
0x4000_0000 +----------------------+
            |   Peripheral region  |   APB1: I2C1/2/3, USART2/3, TIM2-7
            |   (memory-mapped     |   APB2: SPI1, USART1, TIM1/15/16
            |    registers)        |   AHB1/2: GPIOA-H, DMA, RCC
0x5FFF_FFFF +----------------------+
```

Notice the peripheral region: this is *why* registers can be accessed as plain C struct members — the linker/CMSIS system maps a C struct (e.g., `I2C_TypeDef`) directly onto this physical address range.

## 4.7 Processor (ARM Cortex-M4)

The processor fetches instructions, decodes them, and executes them, including special instructions for interrupt handling (via the NVIC — Nested Vectored Interrupt Controller) and low-power modes. The Cortex-M4 adds a hardware FPU and DSP instructions versus the Cortex-M0/M0+, but for driver-writing purposes, what matters most is its **memory-mapped I/O model**: peripherals are accessed with the exact same `LDR` / `STR` instructions used for RAM.

### 4.8 Hardware

The physical silicon — the actual I2C state machine, GPIO pad drivers, clock generation circuitry — sits at the bottom. Firmware never "sees" transistors; it only ever sees registers, which are the *hardware's chosen interface* to firmware.

### 4.9 Full Call Trace Example

Let's trace what happens, layer by layer, when application code calls a (future) driver function:

```
main() calls:
    I2C_MasterTransmit(&hi2c1, 0x50, buf, 4);
        |
        v
[DRIVER LAYER]  i2c_driver.c
    - Validates handle state (idle?)
    - Generates START condition:
          hi2c1->Instance->CR2 |= I2C_CR2_START;
        |
        v
[CMSIS LAYER]  stm32l452xx.h
    - I2C1 macro expands to a pointer:
          #define I2C1  ((I2C_TypeDef *) I2C1_BASE)
    - I2C_TypeDef struct defines CR2 at correct byte offset
        |
        v
[LINKER + MEMORY MAP]
    - I2C1_BASE (e.g., 0x4000_5400) is a real physical address
    - The struct member CR2 lands exactly on the real CR2 register offset
        |
        v
[PROCESSOR]
    - CPU executes STR instruction to that physical address
        |
        v
[HARDWARE]
    - I2C1 peripheral's internal state machine sees CR2.START = 1
    - Physically drives SDA/SCL pins to generate a START condition
```

This trace is the single most important diagram in this lecture: **it shows that a driver function call is nothing mystical — it is a disciplined, layered path from a meaningful function name down to a single hardware-changing instruction.**

---

## 5. Part 3 — CMSIS (Cortex Microcontroller Software Interface Standard)

### 5.1 Why ARM Introduced CMSIS

Before CMSIS, every MCU vendor (ST, NXP, TI, Atmel…) invented its own naming conventions, its own startup code style, its own way of representing interrupt vectors — even though they all used the same ARM Cortex-M core. This meant:

- Code written for one vendor's Cortex-M0 chip could not be recompiled for another vendor's Cortex-M0 chip without significant rewriting.
- Debuggers and IDE tool vendors had to write custom support for every vendor separately.
- Engineers moving between vendors had to relearn low-level conventions from scratch.

ARM introduced **CMSIS** as a **standardized interface layer** so that anything *related to the ARM core itself* (not the vendor's peripherals) looks and behaves the same, everywhere.

### 5.2 CMSIS Core Layer

The CMSIS-Core layer provides standardized access to functionality that is part of the **ARM core itself**, not the vendor's silicon:

- NVIC control functions: `NVIC_EnableIRQ()`, `NVIC_SetPriority()`.
- SysTick timer control: `SysTick_Config()`.
- Core registers and intrinsics: `__NOP()`, `__WFI()`, `__disable_irq()`, `__enable_irq()`.
- Data type definitions consistent across all Cortex-M compilers (`core_cm4.h`).

### 5.3 CMSIS Device Layer

The CMSIS-Device layer is **vendor-specific** (ST provides this for STM32) and defines:

- Register structs like `I2C_TypeDef`, `GPIO_TypeDef`, `RCC_TypeDef`.

- Base addresses of every peripheral instance ( `I2C1_BASE` , `GPIOA_BASE` ).
- Bit definitions ( `I2C_CR1_PE` , `RCC_APB1ENR1_I2C1EN` ).
- The interrupt vector table layout specific to that chip ( `IRQn_Type` enum).

This is the file you will interact with constantly in this course: `stm32l452xx.h` .

## 5.4 Startup Files

CMSIS also standardizes the **shape** of the startup file ( `startup_stm32l452xx.s` ), even though its exact content is vendor-generated. It defines:

- The vector table (an array of function pointers, placed at the very start of Flash).
- `Reset_Handler` , which performs the steps shown in Section 4.5.
- Weak default handlers for every interrupt (so unimplemented interrupts don't crash silently into undefined memory).

## 5.5 Interrupt Vectors

```
Vector Table (conceptual layout, STM32L452 — partial)

Address Offset   Entry
-------------    ----------------------------
 0x00            Initial Stack Pointer value
 0x04            Reset_Handler
 0x08            NMI_Handler
 0x0C            HardFault_Handler
 ...
 0x40            WWDG_IRQHandler
 ...
 0x xx            I2C1_EV_IRQHandler   <-- I2C1 event interrupt
 0x xx            I2C1_ER_IRQHandler   <-- I2C1 error interrupt
```

Every entry is a **function pointer**. When hardware raises an interrupt, the NVIC uses the interrupt number to index into this table and jump straight to the correct handler — this is why it's called a *vectored* interrupt controller (as opposed to a single shared interrupt handler that must poll to figure out the source).

## 5.6 Register Definitions

CMSIS device headers define registers using nested C structs that mirror the exact memory layout of the peripheral:

```
        typedef struct
        {
          __IO uint32_t CR1;    /* Control register 1,       Address offset: 0x00 */
          __IO uint32_t CR2;    /* Control register 2,       Address offset: 0x04 */
          __IO uint32_t OAR1;   /* Own address register 1,   Address offset: 0x08 */
          __IO uint32_t OAR2;   /* Own address register 2,   Address offset: 0x0C */
          __IO uint32_t TIMINGR;/* Timing register,          Address offset: 0x10 */
          /* ... more registers ... */
        } I2C_TypeDef;

        #define I2C1_BASE   (APB1PERIPH_BASE + 0x5400UL)
        #define I2C1        ((I2C_TypeDef *) I2C1_BASE)
```

Because C guarantees struct members are laid out in declared order (with natural alignment), `I2C1->CR2` compiles down to *exactly* the address `I2C1_BASE + 0x04` — no runtime lookup, no overhead, just a direct memory access.

`__IO` is a CMSIS macro expanding to `volatile` , telling the compiler: *"do not optimize away or reorder accesses to this variable — its value can change due to hardware, and every access has a real side effect."*

## 5.7 CMSIS Naming Conventions

| Convention | Example | Meaning |
|---|---|---|
| `PERIPH_BASE` | `I2C1_BASE` | Base address of a peripheral instance |
| `PERIPH_TypeDef` | `I2C_TypeDef` | Struct type describing a peripheral's register layout |
| `PERIPH_REG_FIELD` | `I2C_CR1_PE` | Bitmask for one field inside one register |
| `__IO` / `__I` / `__O` | — | Volatile read-write / read-only / write-only qualifiers |
| `IRQn_Type` enum | `I2C1_EV_IRQn` | Interrupt number used with `NVIC_EnableIRQ()` |

## 5.8 Advantages and Limitations of CMSIS

**Advantages**

- Uniform, self-documenting register access ( `I2C1->CR1` vs a bare hex address).
- Compiler-checked (typos in a struct member name fail to compile; typos in a raw address silently compile).
- Portable across toolchains (GCC, Keil, IAR all support the same CMSIS headers).
- Debugger-friendly — most IDEs can show live register values by name using CMSIS-SVD files.

**Limitations**

- CMSIS gives you **structure**, not **behavior** — it does not sequence register writes correctly for you; that is the driver's job.
- CMSIS does not protect against invalid combinations of bits — you can still misconfigure hardware even while using CMSIS "correctly."
- CMSIS is vendor-authored — you're trusting ST's header accuracy (usually reliable, but errata documents still matter).

---

# 6. Part 4 — Professional Driver Architecture: Industry Comparison

Different organizations have solved the "how do we architect a driver" problem in different ways. Understanding these differences builds your engineering judgement.

## 6.1 STM32Cube HAL Architecture

```
Application
   │
   v
HAL_I2C_Master_Transmit(&hi2c1, addr, buf, len, timeout)
   │
   v
Handle struct: I2C_HandleTypeDef
   ├── Instance (pointer to I2C_TypeDef, i.e., the register block)
   ├── Init (I2C_InitTypeDef — configuration parameters)
   ├── State (HAL_I2C_StateTypeDef — busy/ready/error tracking)
   └── ErrorCode
   │
   v
Direct register access inside HAL source files
```

- Uses a **handle structure** per peripheral instance (so the same code works for I2C1, I2C2, I2C3).
- Heavily uses **callbacks** ( `HAL_I2C_MasterTxCpltCallback` ) for asynchronous completion notification.
- Prioritizes **portability across the entire STM32 family** (F0 to H7) over raw performance.
- Criticized in industry for being large, sometimes slow, and sometimes hiding too much (masking exact timing behavior needed in hard real-time code) — one of the reasons this course teaches bare-metal instead.

## 6.2 Zephyr RTOS Driver Model

```
Application (Zephyr thread)
   │
   v
i2c_transfer(dev, msgs, num_msgs, addr)     <-- generic API, not device-specific
   │
   v
struct device (generic device object)
   ├── name
   ├── config (pointer to const, compile-time configuration)
   ├── data (pointer to mutable runtime state)
   └── api (pointer to struct i2c_driver_api — function pointer table!)
   │
   v
Vendor-specific driver (e.g., i2c_stm32.c) implements the function pointer table
```

- Zephyr's defining idea: the **application never calls a vendor-specific function name at all.** It calls a *generic* API ( `i2c_transfer` ), and a function-pointer table (set at compile/link time via devicetree) dispatches to the actual STM32 (or NXP, or Nordic) implementation.
- This is the clearest real-world example of **interface/implementation separation** — a concept we introduce formally in Part 5.
- Configuration is described in **devicetree** (a data-driven, declarative description of hardware), not hand-written C initialization code.

## 6.3 Linux Kernel Driver Model

```
User-space application
   |
   v  (system call, e.g., open()/read()/write()/ioctl())
Virtual Filesystem (VFS) layer
   |
   v
Character/Block/Network device subsystem
   |
   v
Bus subsystem (I2C core: i2c_add_driver(), i2c_transfer())
   |
   v
Platform-specific I2C controller driver (e.g., i2c-stm32f7.c equivalent)
   |
   v
Hardware registers (accessed via ioremap() memory-mapped I/O)
```

- Linux formalizes an entire **driver model** with buses, devices, and drivers as first-class kernel objects that can be matched at runtime ( `probe()` / `remove()` ).
- User-space applications are **fully isolated** from hardware — they cannot access memory-mapped registers at all (protected by the MMU), unlike bare-metal MCU firmware where everything shares one address space.
- Drivers register themselves with a **subsystem core** (the I2C core), which is conceptually similar to Zephyr's generic API idea, but with a much heavier, dynamic, and process-isolated model appropriate for a full OS.

### 6.4 Automotive Firmware (AUTOSAR-style)

- Automotive firmware (AUTOSAR standard) enforces an extremely rigid **layered architecture**: Application → RTE (Runtime Environment) → Basic Software (BSW, including MCAL — the microcontroller abstraction layer) → Hardware.
- The **MCAL** layer is conceptually identical to what we are building in this course: a thin, direct-register driver layer, standardized by API name (e.g., `I2c_SyncTransmit()` ), but require by the standard to be independent of upper software layers.
- Automotive drivers place extreme emphasis on **determinism, error detection, and diagnostics** (often including redundant safety checks) because of functional safety standards (ISO 26262).

### 6.5 Generic RTOS Environments (FreeRTOS + custom drivers)

- FreeRTOS itself has **no built-in driver model** — it only provides task scheduling and synchronization primitives (queues, semaphores, mutexes).
- Driver design in FreeRTOS projects is therefore **entirely up to the firmware architect** — commonly, drivers use a mutex to protect a shared peripheral, and a semaphore given from an ISR to signal transfer completion to a waiting task.
- This is the model closest to what you, as an embedded driver engineer, will build by hand in professional bare-metal or lightweight-RTOS projects.

### 6.6 Comparison Table

| Aspect | STM32Cube HAL | Zephyr | Linux Kernel | Automotive (AUTOSAR) | Bare-metal + RTOS (this course) |
|---|---|---|---|---|---|
| API style | Vendor-specific function names | Generic API + function pointer dispatch | Subsystem core + registered drivers | Standardized API names (MCAL) | Custom, hand-designed API |
| Configuration | Init structs in C code | Devicetree (declarative) | Device tree / ACPI + kernel config | Static config generated from tooling | Init structs in C code |
| Portability scope | Across STM32 family | Across many silicon vendors | Across architectures, huge scale | Across automotive MCU vendors | Whatever you design it for |
| Concurrency model | Interrupt + optional RTOS | RTOS-native (kernel objects) | Full process/thread isolation, MMU | Statically scheduled, deterministic | Depends on RTOS chosen (or none) |
| Primary goal | Ease of use, broad coverage | Standardization across vendors | Generality, scale, user-space safety | Functional safety, determinism | Learning, control, minimal footprint |

---

## 7. Part 5 — Driver Design Principles

This is the conceptual heart of the lecture. Every principle below will reappear, concretely, when we design the I2C driver.

### 7.1 Abstraction

**Definition:** Presenting *what* an operation does while hiding *how* it is done.

```
Caller sees:                Driver hides:
-----------------           --------------------------------
I2C_MasterTransmit(...)  →   START condition generation
                             Address + R/W bit framing
                             ACK/NACK polling
                             Clock stretching handling
                             TXE/BTF flag polling
                             STOP condition generation
```

Without abstraction, every application would need to reimplement the I2C protocol state machine. With abstraction, the application trusts the driver to "just make the bytes arrive."

## 7.2 Encapsulation

**Definition:** Bundling data (state) and the operations on that data together, and hiding internal representation from outside code.

In C (no classes), encapsulation is achieved by:

- Keeping a peripheral's live state (busy flag, error code, current buffer pointer) inside a `handle` struct.
- Marking internal helper functions `static` so they cannot be called from outside the driver's `.c` file.
- Exposing only a small, curated set of public functions in the driver's `.h` file.

```c
/* i2c_driver.h  (PUBLIC — what the rest of the firmware sees) */
typedef struct I2C_Handle I2C_Handle_t;   /* opaque-ish handle */

I2C_Status_t I2C_Init(I2C_Handle_t *handle, const I2C_Config_t *config);
I2C_Status_t I2C_MasterTransmit(I2C_Handle_t *handle, uint16_t addr,
                                const uint8_t *data, uint16_t len);

/* i2c_driver.c  (PRIVATE — internal helpers, never exposed) */
static void I2C_GenerateStart(I2C_Handle_t *handle);
static I2C_Status_t I2C_WaitFlag(I2C_Handle_t *handle, uint32_t flag);
```

## 7.3 Modularity

**Definition:** Splitting the system into independent, self-contained units (modules), each with a single, well-defined responsibility.

A modular I2C driver, for example, is a **separate module** from the GPIO driver, even though I2C needs GPIO pins configured (for SDA/SCL). The I2C driver *calls* the GPIO driver's public API — it does not reach into GPIO registers itself.

## 7.4 Portability

Already discussed in Section 3.8 — portability is *achieved through* abstraction and modularity, not a separate technique. A driver is portable when hardware-specific code is isolated behind a stable interface, so only that isolated part needs to change when the target chip changes.

## 7.5 Loose Coupling

**Definition:** Minimizing how much one module needs to know about another module's internal details.

- **Tightly coupled (bad):** the UART driver directly reads a global variable owned by the I2C driver.
- **Loosely coupled (good):** the UART driver calls a documented, public function from the I2C driver, or better, doesn't depend on it at all.

Loose coupling is what allows you to **replace or upgrade one driver without breaking others.**

## 7.6 High Cohesion

**Definition:** Everything inside a single module should be closely related to that module's one purpose.

A "high cohesion" I2C driver file contains *only* I2C-related logic. A "low cohesion" (bad) design might dump GPIO configuration, delay functions, and I2C logic into a single giant file — this is a common beginner mistake.

## 7.7 Code Reuse

Achieved by writing **generic, parameterized** functions rather than one-off, hardcoded ones. Example: a single `I2C_WaitFlag()` helper function used everywhere a flag must be polled, instead of copy-pasted polling loops scattered throughout the driver.

## 7.8 Configuration Structures

**Definition:** A struct that bundles all user-selectable parameters for initializing a peripheral, instead of a function with many individual arguments.

```c
typedef struct
{
    uint32_t   ClockSpeed;      /* Desired SCL frequency in Hz         */
    uint8_t    OwnAddress;      /* Own address if used as slave        */
    uint8_t    AddressingMode;  /* 7-bit or 10-bit addressing          */
    uint8_t    DutyCycle;       /* Fast-mode duty cycle option         */
} I2C_Config_t;
```

**Why not just pass parameters directly?** Because functions with 6+ parameters are error-prone (easy to swap argument order), hard to extend (adding a new option breaks every call site), and hard to read at the call site. A config struct scales gracefully — you can add a new field without breaking existing code that uses designated initializers.

## 7.9 Handle Structures

**Definition:** A struct representing *one instance* of a peripheral, bundling together its register pointer, its configuration, and its live runtime state.

```c
typedef struct
{
    I2C_TypeDef      *Instance;   /* Pointer to the register block, e.g. I2C1 */
    I2C_Config_t      Config;     /* User configuration                       */
    I2C_State_t       State;      /* Runtime state machine: idle/busy/error   */
    uint8_t          *pTxBuffer;
    uint16_t          TxLen;
} I2C_Handle_t;
```

Handles are *why* the same driver source code can control I2C1, I2C2, and I2C3 simultaneously — you simply create three handle instances, each pointing at a different `Instance`.

## 7.10 Callbacks

**Definition:** A function pointer supplied by the application, which the driver invokes when a specific event occurs (e.g., transfer complete, error detected).

Callbacks allow the driver to **notify** upper layers without the driver needing to know anything about what the application wants to do in response — this is a form of **inversion of control**, keeping the driver generic and the application in charge of behavior.

## 7.11 State Machines

Every non-trivial peripheral driver is, internally, a **state machine**. For I2C master transmission, a simplified state machine looks like:

```
   [IDLE]
      |  I2C_MasterTransmit() called
      v
[GENERATE START]
      |  SB flag set by hardware
      v
[SEND ADDRESS]
      |  ADDR flag set (ACK received)
      v
[TRANSMIT DATA]  <--------+
      |  TXE flag set      | more bytes remain
      +--------------------+
      |  all bytes sent, BTF flag set
      v
[GENERATE STOP]
      |
      v
   [IDLE]
```

Tracking state explicitly (rather than assuming "it always works") is what allows a driver to **detect and reject invalid operations** — e.g., refusing a second `I2C_MasterTransmit()` call while one is already in progress.

## 7.12 Layer Separation

Already covered architecturally in Part 2 — reinforced here as a *design principle*: never let a lower layer depend on a higher layer. CMSIS must never call into a driver. A driver must never call into application code directly (only via callbacks, which is a controlled, explicit exception).

## 7.13 Error Handling

Professional drivers **never silently fail**. Every public API function returns a status code:

```
typedef enum
{
    I2C_OK = 0,
    I2C_ERROR_TIMEOUT,
    I2C_ERROR_NACK,
    I2C_ERROR_BUS_BUSY,
    I2C_ERROR_INVALID_PARAM
} I2C_Status_t;
```

This allows calling code to make informed decisions (retry, log, alert the user) instead of the driver hanging forever or corrupting memory when hardware misbehaves.

### 7.14 API Design Philosophy

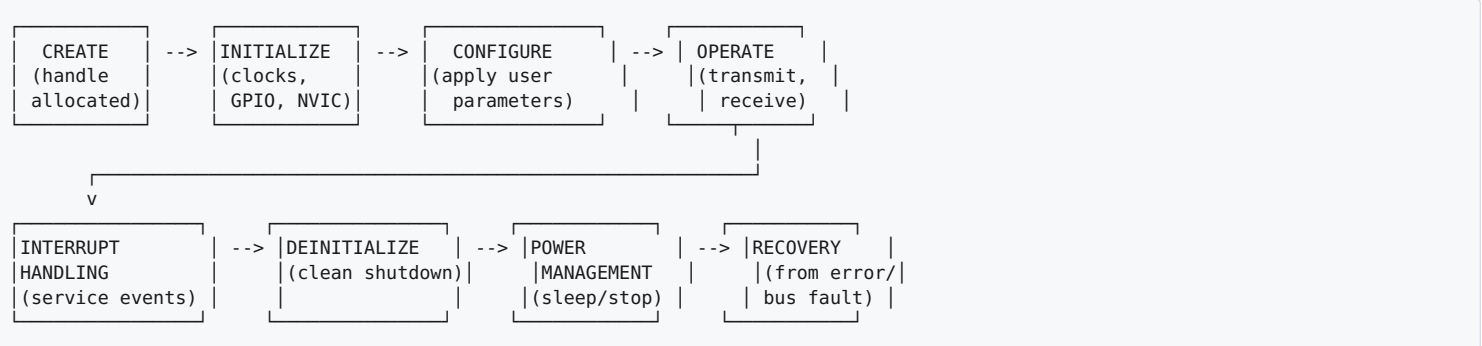
Good embedded API design follows these rules of thumb:

1. **Consistent naming** — every function starts with the module prefix (`I2C_...`).
2. **Consistent parameter ordering** — handle first, always.
3. **Return codes, not `void`**, for anything that can fail.
4. **No hidden global state** — everything needed to operate is inside the handle.
5. **Symmetry** — if there's an `Init`, there should be a `DeInit`. If there's an `Enable`, there should be a `Disable`.

---

## 8. Part 6 — Driver Lifecycle

Every driver you write in this course — I2C, SPI, UART, Timer — passes through the same eight lifecycle stages.




### 8.1 Driver Creation

Allocating (usually statically, in embedded systems) the handle structure that will represent one instance of the peripheral. In bare-metal firmware, this is typically a global or file-scope struct — dynamic `malloc()` is generally avoided for determinism and to prevent heap fragmentation on memory-constrained MCUs.

### 8.2 Initialization

Bringing supporting infrastructure online: enabling the peripheral's clock (via RCC registers), configuring the required GPIO pins into their alternate function mode, and optionally enabling the associated NVIC interrupt line. Initialization does **not** yet apply the user's desired protocol parameters — it prepares the ground for configuration.

### 8.3 Configuration

Applying the actual protocol parameters from the `I2C_Config_t` struct: clock speed (which requires calculating and writing `TIMINGR` values), addressing mode, own address, etc. This is where configuration structures (Section 7.8) are consumed.

### 8.4 Operation

The peripheral is now used for its intended purpose: transmitting or receiving data. This stage is governed by the state machine discussed in Section 7.11, and is where the majority of driver code (and majority of subtle bugs) lives.

### 8.5 Interrupt Handling

If interrupt-driven (rather than polled) operation is used, the driver must implement an ISR (Interrupt Service Routine) that reacts to hardware events — received byte, transfer complete, error condition — quickly and safely (ISRs should do minimal work and defer heavy processing, typically via a callback or a flag checked by the main loop / RTOS task).

### 8.6 De-initialization

The reverse of initialization: disabling the peripheral, releasing GPIO pins back to a default/analog state (to save power), and disabling the associated clock — important for power-sensitive applications and for cleanly handing the peripheral to a different configuration later.

### 8.7 Power Management

Many MCUs, including the STM32L4 series (which is specifically a **low-power** line), support multiple sleep modes (Sleep, Stop 0/1/2, Standby, Shutdown). A professional driver must know how its peripheral behaves across these modes — does it retain configuration in Stop mode? Does it need to be reinitialized after Standby? This is a frequent source of subtle real-world bugs.

### 8.8 Recovery

Real hardware occasionally enters an unexpected state: a slave device stretches the clock forever, a bus lockup occurs (SDA stuck low), or noise on the line causes a spurious start condition. Professional drivers implement **recovery procedures** — for I2C, a classic recovery technique is manually toggling SCL to force a stuck slave to release SDA, then reinitializing the peripheral. We will implement real I2C bus-recovery logic later in this course.

---

## 9. Part 7 — Mini Driver Example: Designing (Not Yet Coding) an LED Driver

Before writing a single line of code, a professional driver engineer goes through a **design process**. We walk through that process now for the simplest possible peripheral — a GPIO-driven LED — specifically so the *process* is clear before we apply it to I2C, which has far more moving parts.

### 9.1 Step 1 — Define the Responsibility

*"This driver controls the on-board LED's on/off/toggle state, and nothing else."* Writing this sentence down first prevents scope creep (e.g., accidentally also handling a button in the same module — low cohesion, Section 7.6).

### 9.2 Step 2 — Identify the Hardware Facts

- The LED is wired to a specific GPIO pin (e.g., PA5 on the NUCLEO-L452RE, the standard "LD2" user LED).
- That GPIO pin belongs to a specific GPIO port peripheral (`GPIOA`), which has its own clock enable bit in RCC.
- The pin must be configured as a **general-purpose output**.

### 9.3 Step 3 — Define the Public API (Before Writing Internals)

```
/* led_driver.h */
typedef struct
{
    GPIO_TypeDef *Port;
    uint16_t      Pin;
} LED_Handle_t;

void LED_Init(LED_Handle_t *led, GPIO_TypeDef *port, uint16_t pin);
void LED_On(LED_Handle_t *led);
void LED_Off(LED_Handle_t *led);
void LED_Toggle(LED_Handle_t *led);
```

**Design reasoning:**

- A **handle struct** (Section 7.9) is used even for something this simple, so the same driver source works for *any* LED on *any* pin — not just the on-board one. This is portability and reuse in action.
- Four small, symmetric functions (Section 7.14) rather than one function with a mode parameter (`LED_SetState(led, MODE_ON)`), because separate functions read more clearly at call sites and are harder to misuse.
- No configuration struct is needed here (contrast with I2C) because there is genuinely nothing to configure beyond which pin — this shows that **you apply exactly as much architecture as the peripheral warrants, no more, no less.** Over-engineering a trivial driver is itself a design mistake.

### 9.4 Step 4 — Decide What the Driver Depends On

The LED driver depends on the **GPIO driver's** public API (to configure pin mode) — it must **not** touch `RCC->AHB2ENR` or `GPIOx->MODER` directly itself. This enforces modularity (Section 7.3) and loose coupling (Section 7.5): if the GPIO driver's internal register sequence changes later (e.g., to add pull-up/pull-down configuration), the LED driver does not need to change at all.

### 9.5 Step 5 — Sketch the Function Call Hierarchy

```
Application
   |
   v
LED_On(&led)
   |
   v
GPIO_WritePin(led.Port, led.Pin, GPIO_PIN_SET)   <-- calls into GPIO driver, not raw registers
   |
   v
led.Port->BSRR = (1U << led.Pin)                 <-- only the GPIO driver touches this
```

This tiny example already contains every architectural idea used by our future I2C driver: handle structs, a clean public API, dependency on a lower driver rather than raw registers, and a call hierarchy that can be drawn and reasoned about before any code exists.

---

## 10. ASCII Diagram Reference Sheet

For quick review, here are the key diagrams from this lecture collected together.

### 10.1 Driver Hierarchy (General Form)

```
              Application
                  |
                  v
     ┌──────────────┴──────────────┐
     |      Peripheral Driver      |   <-- I2C, SPI, UART, Timer...
     |  (public API + handle)      |
     └──────────────┬──────────────┘
                    |  (may depend on)
                    v
     ┌──────────────┴──────────────┐
     |   Supporting Driver(s)      |   <-- GPIO driver, DMA driver
     └──────────────┬──────────────┘
                    |
                    v
     ┌──────────────┴──────────────┐
     |          CMSIS              |
     └──────────────┬──────────────┘
                    |
                    v
              Hardware
```

### 10.2 Register Access Path

```
C code:  I2C1->CR1 |= I2C_CR1_PE;
              |
              v
  I2C1  ==  ((I2C_TypeDef*) 0x40005400)
              |
              v
  CR1 offset == +0x00, so address == 0x40005400
              |
              v
  Compiler emits:  LDR r0, =0x40005400
                   LDR r1, [r0]
                   ORR r1, r1, #0x1
                   STR r1, [r0]
              |
              v
  Physical I2C1 peripheral enable circuit activates
```

### 10.3 Interrupt Flow (General Form)

```
Hardware event occurs (e.g., byte received)
        |
        v
Peripheral sets a status flag + raises interrupt signal
        |
        v
NVIC checks: is this interrupt enabled? what priority?
        |
        v
NVIC vectors CPU to the matching ISR (e.g., I2C1_EV_IRQHandler)
        |
        v
ISR (inside the driver) reads status, clears flag, updates handle state
        |
        v
ISR optionally invokes a user-supplied callback
        |
        v
CPU returns from interrupt, resumes previous execution
```

---

# 11. Common Mistakes

## 11.1 Beginner Mistakes

| Mistake | Why It's a Problem | Fix |
|---|---|---|
| Accessing peripheral registers directly from `main()` | Breaks ownership, portability, and testability (Section 3.6) | Always go through a driver API |
| Forgetting `volatile` when hand-rolling register access outside CMSIS | Compiler may cache the value in a register and never re-read hardware | Use CMSIS `__IO` types; never bypass them |
| One giant `.c` file containing GPIO + I2C + UART logic | Low cohesion (Section 7.6), impossible to reuse or test independently | One module per peripheral, own `.h` / `.c` pair |
| Using magic numbers instead of CMSIS bit-defines ( `I2C1->CR1 \|= (1 << 0);` ) | Unreadable, error-prone, breaks if bit position ever changes | Always use `I2C_CR1_PE` -style CMSIS macros |
| No return codes — functions are `void` and assumed to "just work" | Silent failures are undebuggable in the field | Every fallible function returns a status enum (Section 7.13) |
| Busy-waiting forever on a status flag with no timeout | A stuck slave or noisy bus causes a permanent hang | Always bound polling loops with a timeout and return an error |

## 11.2 Professional-Level Mistakes

| Mistake | Why It's a Problem | Fix |
|---|---|---|
| Designing the handle struct *after* writing the operation code | Leads to scattered global state instead of clean encapsulation | Design the handle and public API first (Part 7 process) |
| Over-abstracting a trivial peripheral (adding callbacks/state machines to something like a single static LED) | Wastes engineering time, adds needless complexity | Match architecture to actual complexity (Section 9.3 reasoning) |
| Ignoring power-mode interaction (Section 8.7) — peripheral misbehaves after Stop mode | Field bugs that are very hard to reproduce in the lab | Explicitly test and document behavior across all supported low-power modes |
| No bus-recovery strategy for I2C | A single glitched transaction can permanently hang the bus until power-cycle | Implement recovery per Section 8.8 |
| Mixing interrupt-context code and main-loop code without proper synchronization | Race conditions, corrupted shared state | Use volatile flags / proper synchronization primitives; keep ISRs minimal |

### 11.3 Debugging Approach for Driver Issues

1. **Check the clock first.** The single most common "nothing works" bug in bare-metal STM32 development is forgetting to enable the peripheral's clock in RCC — the peripheral registers may even read back garbage or reset values if unclocked.
2. **Check GPIO alternate function mapping.** Wrong AF number silently routes the pin to the wrong internal peripheral.
3. **Use the debugger's peripheral register view** to inspect live register values against expected ones (this is where CMSIS + SVD files pay off).
4. **Add a timeout to every wait loop** while debugging, even temporarily, so a hang becomes a reported error instead of a frozen board.
5. **Isolate**: test the driver in the smallest possible standalone program before integrating with the rest of the firmware.

---

## 12. Part 8 — Interview Questions

### 12.1 Basic (Questions 1–10)

1. What is a device driver, and how does it differ from just "code that touches hardware"?
2. Why should application code never access peripheral registers directly?
3. What is the purpose of the `volatile` keyword in register access, and what happens if it's omitted?
4. What are the main layers of the embedded software stack, from application down to hardware?
5. What is CMSIS, and why did ARM introduce it?
6. What is the difference between CMSIS-Core and CMSIS-Device?
7. What does a linker script define, and why doesn't it "run" like code?
8. What are the responsibilities of startup code before `main()` executes?
9. What is a handle structure, and why is it useful for supporting multiple peripheral instances?
10. What is the difference between firmware, driver, and application?

### 12.2 Intermediate (Questions 11–20)

11. Explain the difference between abstraction and encapsulation with a driver example.
12. Why does STM32Cube HAL use configuration structs instead of long parameter lists?
13. Describe the driver lifecycle stages and why de-initialization matters.
14. What problem does a callback solve that a return value cannot?
15. Explain loose coupling vs tight coupling using a GPIO/I2C example.
16. Why is code that busy-waits on a hardware flag without a timeout dangerous?
17. How does the NVIC use the interrupt vector table to service an interrupt?
18. What's the difference between a peripheral's initialization stage and its configuration stage?
19. Why might a driver need a bus-recovery procedure, and when would it run?
20. Explain why high cohesion and low coupling are both necessary — what happens if you have one without the other?

### 12.3 Advanced (Questions 21–27)

21. Compare and contrast the STM32Cube HAL driver model with Zephyr's generic device/API model.
22. In the Linux kernel driver model, how does user-space isolation from hardware change how a driver must be architected compared to bare-metal firmware?
23. Why does AUTOSAR enforce such a rigid layered architecture (Application/RTE/BSW/MCAL), and what does this buy the automotive industry?
24. What tradeoffs are involved in choosing static (compile-time) handle allocation vs dynamic (`malloc`) allocation for driver handles in a safety-critical system?
25. Explain how low-power modes (like STM32L4's Stop modes) can break naive driver assumptions, and how a driver should account for this.
26. Design a strategy for a driver to safely support both interrupt-driven and polled operation modes without duplicating logic.
27. What architectural changes would you make to a bare-metal driver if it needed to become thread-safe under an RTOS?

### 12.4 Scenario-Based (Questions 28–30)

28. **Scenario:** A colleague's I2C driver works fine in testing but occasionally hangs forever in the field when a slave device misbehaves. What architectural weaknesses would you look for, and what would you add?
29. **Scenario:** You are asked to port an I2C driver written for STM32L452 to a completely different vendor's Cortex-M4 MCU. Walk through which parts of the driver should require zero changes, and which parts must be rewritten, based on the architecture from this lecture.
30. **Scenario:** Two RTOS tasks both need to send data over the same I2C bus at different times. Using the design principles from this lecture (state machine, handle, encapsulation), describe how you would prevent one task's transfer from corrupting the other's.

---

## 13. Part 9 — Exercises

### 13.1 Theory Exercises

1. In your own words (do not copy this lecture), explain why a driver is more than "code that writes to registers."
2. Draw the embedded software stack (Section 3.3) from memory, and explain the direction of data flow vs. interrupt flow.
3. Explain, without looking back at Part 5, the difference between abstraction, encapsulation, and modularity.
4. List all eight stages of the driver lifecycle in order, and give one real consequence of skipping each one.

### 13.2 Coding Exercises

5. Write the `LED_Handle_t` struct and function prototypes from Part 7 into an actual `led_driver.h` file, with proper header guards.
6. Write a `GPIO_Config_t` configuration struct (Section 7.8 style) for a generic GPIO driver, including at minimum: mode, output type, pull-up/down, and speed.
7. Write an `I2C_Status_t` enum (Section 7.13 style) with at least six meaningful error codes you predict an I2C driver will need, based only on this lecture (we will refine this in the I2C lecture).
8. Write a skeleton (prototypes only, no implementation) for an I2C driver's public header file, based on the lifecycle stages in Part 6.

### 13.3 Debugging Exercises

9. A junior engineer's driver code reads: `I2C1->CR1 |= 1;` with no comment. Identify at least three problems with this line based on this lecture, and rewrite it correctly.
10. A driver function polls a status flag in a `while(1)` loop with no timeout and no error path. Rewrite it to include a bounded timeout and a proper `I2C_Status_t` return value.
11. A team's I2C driver and UART driver both `#include` a shared `globals.h` containing a single shared `uint8_t buffer[64]`, used by both drivers. Identify the coupling/cohesion problem and propose a fix.

### 13.4 Driver Modification Exercises

12. Extend the Part 7 LED driver design (do not write full code yet) to support a "blink N times" operation. Decide: does this require a state machine? Justify your answer using Section 7.11's reasoning.
13. Redesign the Part 7 LED driver's public API to also support active-low LEDs (where writing 0 turns the LED on) without changing any call site in application code.
14. Propose how you would add a callback to the LED driver that fires after a timed toggle completes, and explain which lifecycle stage this callback logically belongs to.
15. Given the industry comparison table in Section 6.6, propose which single idea from Zephyr's driver model (generic API + function pointer table) you would introduce into our bare-metal driver design, and explain one advantage and one disadvantage of doing so.

---

## 14. Part 10 — Mini Assignment

**Assignment: Design (Architecture Only) a GPIO Driver**

You will **not** submit working register-level code for this assignment — the goal is architecture, mirroring the process from Part 7.

**Deliverables:**

1. **Responsibility statement** — one or two sentences defining exactly what your GPIO driver will and will not do.
2. **Configuration structure** (`GPIO_Config_t`) — list every field you believe a professional GPIO driver needs (mode, pull, speed, alternate function, output type, etc.), and briefly justify each field's presence.
3. **Handle structure** (`GPIO_Handle_t`) — show what it contains and explain why each member belongs there.
4. **Public API function list** — function signatures only, following the naming and parameter-ordering conventions from Section 7.14.
5. **Function call hierarchy diagram** (ASCII, like Section 9.5) showing how your GPIO driver's `Init` function would eventually reach real hardware, referencing CMSIS structures conceptually (you do not need real STM32L452 register names yet — that is covered in the next lecture).
6. **One paragraph** explaining how your design would need to change (if at all) to support both input pins with interrupt-on-change and output pins, referencing the driver lifecycle from Part 6.

**Grading criteria (self-assessment checklist):**

- [ ] Does every struct field have a clear, justified purpose (no "just in case" fields)?
- [ ] Is there a clean separation between configuration (compile-time-ish, rarely changing) and state (runtime, frequently changing)?
- [ ] Does the API follow consistent naming and handle-first parameter ordering?
- [ ] Would this design still work cleanly if the firmware needed to control 8 GPIO pins across 3 different ports simultaneously?
- [ ] Is there anything in your design that directly touches a register, rather than describing configuration intent? (There should not be — that's the next lecture's job.)

---

# 15. Summary

## 15.1 What We Covered

- A device driver is an architectural boundary, not just "code that touches hardware" — it provides ownership, a stable API, and hardware-detail hiding.
- The embedded software stack flows from Application → Middleware → Drivers → CMSIS → Startup → Linker → Processor → Hardware, and we traced a full function call through every layer.
- CMSIS standardizes ARM-core-related access (Core layer) and vendor-specific register/interrupt definitions (Device layer), giving drivers a compiler-checked, portable foundation.
- Different industries solve driver architecture differently — STM32Cube HAL (ease of use), Zephyr (generic API + function-pointer dispatch), Linux (subsystem + process isolation), AUTOSAR (rigid layering for safety) — but all share the same underlying goal: isolate hardware detail behind a stable interface.
- Professional drivers are built from a consistent toolbox of principles: abstraction, encapsulation, modularity, portability, loose coupling, high cohesion, configuration structs, handle structs, callbacks, state machines, and disciplined error handling.
- Every driver follows the same eight-stage lifecycle: Create → Initialize → Configure → Operate → Interrupt Handling → De-initialize → Power Management → Recovery.
- We designed (not coded) a minimal LED driver end-to-end, demonstrating that even the simplest peripheral benefits from handle-based, API-first design — and that over-engineering trivial peripherals is itself a mistake to avoid.

## 15.2 Key Takeaways

> ***The registers are not the driver. The architecture around the registers is the driver.***

> ***Every design decision in a professional driver — the handle struct, the config struct, the state machine, the callback — exists to solve a specific, real engineering problem, not to look sophisticated.***

> ***The lifecycle (Create → Init → Configure → Operate → Interrupt → Deinit → Power → Recovery) is the skeleton for every driver in this course, including the I2C driver we build starting next lecture.***

## 15.3 What Comes Next

**Lecture 2** will apply everything from this lecture directly to the STM32L452's I2C peripheral: its internal hardware architecture (from RM0394), the exact registers involved (CR1, CR2, OAR1/2, TIMINGR, ISR), and the beginning of our real, production-quality `i2c_driver.h` / `i2c_driver.c` implementation — starting with the configuration structure and handle structure designed using the exact principles from Part 5 of this lecture.

---